

Test Plan Documentation

<https://github.com/DanielJ0131/interactive-house>

Interactive House – SG2 Units

Revision History.....	1
Relation between Requirements, Design and Tests.....	2
Test Plan.....	3
1.1 General.....	4
1.2 Mobile.....	4
1.2.1 Scope.....	4
1.2.2 Testing Approach & Tools.....	4
1.2.3 Verification & Validation (V&V).....	5
1.2.4 Test Examples.....	5
1.3 Web.....	5
1.3.1 Scope.....	6
1.3.2 Testing Approach & Tools.....	6
1.3.3 Verification & Validation.....	6
1.3.4 Test Examples.....	7
Figures.....	8
Mobile.....	9
Figure 1: Mobile Testing Structure in Visual Studio Code.....	9
Figure 2: Mobile Total Tests.....	10
Figure 3: Mobile Coverage.....	10
Figure 4: Mobile Coverage Ai, Hub, Music (Main tabs).....	11
Web.....	11
Figure 5: Web Test Files.....	12
Figure 6: Web Total Tests.....	12
Figure 7: Web Coverage.....	12

Revision History

Date	Version	Description	Author
15/Mar/26	1.0	Initial Test Plan for React Native/Expo Using Jest (1. General and 2. Mobile)	Daniel Jönsson & Ali Daoud
15/Apr/26	1.1	Add UI/Unit Tests to Mobile App and Coverage. Add Table of Contents and New Figures (1, 2, 3)	Daniel Jönsson
8/May/26	1.2	Add Web Testing Section, Test Examples, Coverage Results. Add Update to all Mobile Test Figures. Went from 178 tests to 248 tests. Figures (2, 3, 4, 5, 6, 7, 8) Updated or new	Patrick Nordahl & Daniel Jönsson
25/May/26	1.3	Add Github Link, Add more Web Tests, Update Figures and add Table for relation between requirements, design and tests.	Daniel Jönsson & Patrick Nordahl

Relation between Requirements, Design and Tests

Requirement Item	Design Item	Corresponding Tests
R1 (General User Authentication), R19 (Web Authentication Integration), R8 (Basic Access Control), R23 (Web Route Protection).	D2 (Web User Authentication), D5 (Mobile Login/Signup).	The Mobile and Web test plan covers signInWithEmailAndPassword functions, valid user identification, error codes, and onAuthStateChanged tracking. The Web test plan covers validation errors for missing fields, Firebase account creation, session cookies, and redirection to the /hub page.
R3 (Display device list), R5 (Send control commands), R20 (Web Device Synchronization), R21 (Web Hub Control Panel).	D3 (Web Device Management Interface), D4 (Web Hub Control Panel), D6 (Mobile Control Devices in Device Hub).	The Mobile and Web test plan uses Jest to mock Firestore onSnapshot listeners to verify the UI reflects database changes. It also verifies that toggling a UI switch sends the correct update command (like turning a light "off") to the database.
R2 (Connect to House	D7 (Mobile Database	

Server), R7 (Handle connection loss), R22 (Web Connection State Handling).	Connection).	Mobile testing implements a withTimeout test to verify the app provides immediate feedback if the server is unreachable.
--	--------------	--

Test Plan

1.1 General

In our role as the Units subgroup, we are responsible for the mobile app and web interface that communicates with the server and database. We will test the data exchange between our unit and the database to ensure commands are sent accurately. In line with SDG 3 and SDG 10, we will test the interface to ensure it supports users with high grades of disabilities, emphasizing self-control and independence. The tests are also made to save time, so we do not have to debug everything ourselves.

1.2 Mobile

1.2.1 Scope

Our testing focuses on ensuring that our React Native application can successfully: Receive and render dynamic User Interfaces (UIs) distributed by the server and database from the observed device states in the Arduino Smart Home, such as data from the temperature sensor or control house devices, such as toggling the light "ON" or "OFF" through the database. We will also verify that the UI adapts correctly to the mobile phone's touch screen or alternative inputs like speech recognition.

1.2.2 Testing Approach & Tools

We will use the "Jest Framework" to perform both Unit and UI Testing on individual React Native components and logic functions. We will also use the Expo Go environment to manually validate that the touch-screen interactions (buttons, gestures) work as intended on mobile hardware.

All tests run locally using: "npm test"

The test setup is configured using Jest with Expo support. The following development dependencies are used for testing: jest, jest-expo, @testing-library/react-native, react-test-renderer, @types/jest.

jest.setup.ts is used for environment setup and required mocks. Test files located

in the `__tests__` directory. The testing environment is isolated from production configuration files.

1.2.3 Verification & Validation (V&V)

For the verification part, we will use the Jest UI/Unit Test Framework to verify that our internal logic meets the requirements defined in our Requirement Document.

For the validation part, we will assess if the final mobile interface successfully supports users with high grades of disabilities.

1.2.4 Test Examples

Authentication and Access Control

Because our project focuses on a smart home control center context, ensuring only authorized users can access the system is a primary requirement.

We test that the `signInWithEmailAndPassword` function correctly identifies valid users and provides specific error codes for invalid attempts.

To support the anytime and everywhere nature of ubiquitous computing, we implemented a `withTimeout` test. This verifies that the app provides immediate feedback if the house server is unreachable, maintaining the user's sense of self-control and independence.

We verify `onAuthStateChanged` to ensure the app accurately tracks whether a user is currently logged in.

Database Observation and Device Control

A major purpose of this project is to study how to communicate with and observe house devices. Our Firestore tests in mobile validate this interaction.

We use Jest here to mock `onSnapshot` listeners. This ensures that the mobile unit correctly reflects the status of devices as they change in the database, allowing the user to observe the house remotely.

We also test the control aspect of the project by verifying that toggling a switch in our React Native UI sends the correct update command like turning a light "off" to the database. For the error handling, we test how the app reacts to missing device

configurations or database permission errors.

1.3 Web

1.3.1 Scope

Our web testing focuses on verifying that the Next.js web interface can load the Firebase configuration correctly and support authentication-related user interactions in the browser environment. The tests mainly cover login, signup, Firebase authentication, and Firestore user document creation developed within the SG2 Units subgroup.

The web interface is also tested to ensure that users are redirected to the smart house hub after successful authentication.

1.3.2 Testing Approach & Tools

We use the Jest Framework together with React Testing Library to perform Unit and UI Testing for the web application.

All tests run locally using: “npm test”

The tests are executed in isolated environments using mocked Firebase services to avoid communication with the production database during testing.

The following areas are currently tested:

- Firestore configuration exports
- Login validation and authentication
- Signup validation and user creation
- Firestore document creation
- User redirection after successful authentication
- Friendly error handling for Firebase authentication errors

Coverage reports are also generated to evaluate tested statements, branches, functions, and lines within the web application.

1.3.3 Verification & Validation

For the verification part, we use Jest Unit and UI tests to verify that the authentication flow and Firebase configuration behave according to the requirements defined in our Requirement Documentation.

For the validation part, we manually validate that users can successfully log in, create accounts, and navigate to the smart house hub through the web interface.

Since the SG2 Units subgroup is mainly responsible for the client-side web and mobile interface, the implemented web tests focus on frontend authentication and Firebase-related functionality.

More extensive system-level and integration testing is documented separately in the SG4 subgroup Test Plan documentation.

1.3.4 Test Examples

Firestore Configuration

The Firestore configuration tests verify that the application correctly exports valid Firestore authentication, Firestore, and application instances.

Firestore modules are mocked during testing to isolate the test environment from the production Firestore configuration.

Authentication and Access Control

The login tests verify that:

- Validation errors are shown when required fields are missing
- Users can successfully sign in using Firestore Authentication
- Users are redirected to the /hub page after successful login
- Authentication session cookies are created

Firebase authentication errors are translated into user-friendly messages

The signup tests verify that:

- Password mismatch validation works correctly

- Firebase accounts can be created successfully

- User profile information is updated correctly

- Firestore user documents are stored successfully

- Users are redirected to the /hub page after successful signup

- Firebase signup errors are displayed as readable messages

Figures

Mobile

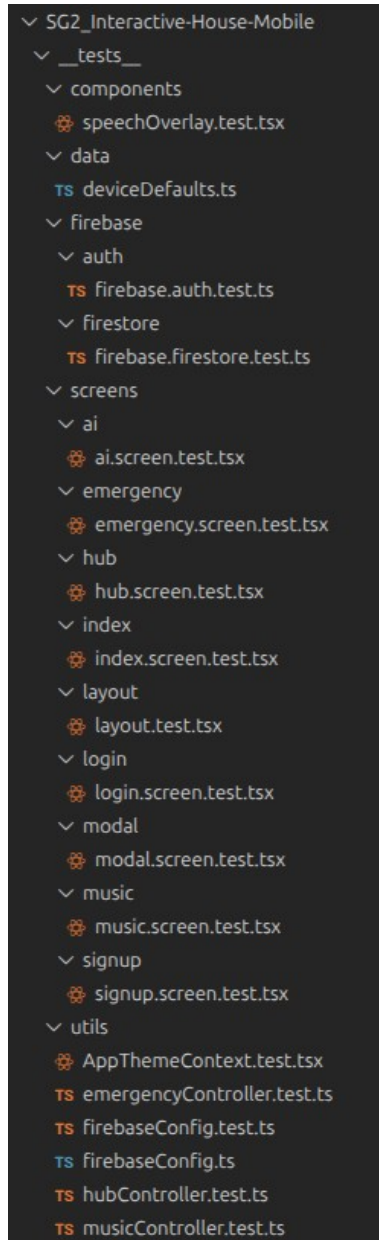


Figure 1: Mobile Testing Structure in Visual Studio Code

```

> interactive-house-mobile@1.0.0 test
> jest

PASS   tests_/utils/musicAudio.web.test.ts
PASS   tests_/components/speechOverlay.test.tsx
PASS   tests_/screens/index/index.screen.test.tsx
PASS   tests_/screens/modal/modal.screen.test.tsx
PASS   tests_/utils/musicAudio.helpers.test.ts
PASS   tests_/utils/AppThemeContext.test.tsx
PASS   tests_/firebase/auth/firebase.auth.test.ts
PASS   tests_/utils/GuestContext.test.tsx
PASS   tests_/utils/musicAudio.native.test.ts
PASS   tests_/screens/layout/layout.test.tsx
PASS   tests_/screens/emergency/emergency.screen.test.tsx
PASS   tests_/utils/hubController.test.ts
PASS   tests_/utils/musicAudio.test.ts
PASS   tests_/utils/musicController.test.ts
PASS   tests_/firebase/firestore/firebase.firestore.test.ts
PASS   tests_/utils/emergencyController.test.ts
PASS   tests_/data/deviceDefaults.test.ts
PASS   tests_/utils/firebaseConfig.test.ts
PASS   tests_/screens/ai/ai.screen.test.tsx
PASS   tests_/screens/login/login.screen.test.tsx
PASS   tests_/screens/signup/signup.screen.test.tsx
PASS   tests_/screens/hub/hub.screen.test.tsx (5.573 s)
PASS   tests_/screens/music/music.screen.test.tsx (6.314 s)

Test Suites: 23 passed, 23 total
Tests:       248 passed, 248 total
Snapshots:   0 total
Time:        6.939 s
Ran all test suites.

```

Figure 2: Mobile Total Tests

All files

89.62% Statements 1278/1417 81.68% Branches 887/988 87.5% Functions 245/280 90.97% Lines 1219/1348

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
app	92.56% 112/121	90.36% 75/83	90% 36/40	92.24% 107/116
app(auth)	93.43% 128/137	85.71% 78/91	87.5% 28/32	96.15% 125/130
app(tabs)	85.75% 668/779	77.95% 495/635	86.92% 133/153	87.48% 643/735
components	93.33% 84/90	91.3% 84/92	100% 8/8	94.04% 79/84
data	100% 1/1	100% 0/0	100% 0/0	100% 1/1
utils	95.84% 277/289	86.2% 75/87	85.1% 40/47	96.35% 264/274

Figure 3: Mobile Coverage

Statements are the individual instructions that make a computer perform an

action. A single line of code might contain one statement, or it could contain several.

Branches refer to the decision points in the code anywhere the control flow can branch off in different directions. This usually involves if/else statements, switch cases, or ternary operators (? :).

Functions is the simplest metric. It tracks whether each function or method defined in the code was called at least once during the tests.

Lines measure the physical lines of code in your source file that were executed.

File ▲	Statements ▾	Branches ▾	Functions ▾	Lines ▾
ai.tsx	 89.74%	70/78 87.5%	49/56 92.3%	12/13 93.05%
hub.tsx	 86.63%	214/247 80%	200/250 90.19%	46/51 89.13%
music.tsx	 84.58%	384/454 74.77%	246/329 84.26%	75/89 85.68%

Figure 4: Mobile Coverage Ai, Hub, Music (Main tabs)

Web

```
PASS  __tests__/proxy/middleware.test.ts
PASS  __tests__/hooks/useSpeech.test.tsx
PASS  __tests__/screens/emergency/emergency.screen.test.tsx
PASS  __tests__/components/floatingSpeech.test.tsx
PASS  __tests__/utils/guestSession.test.ts
PASS  __tests__/app/layout.test.tsx
PASS  __tests__/firebase/firestore/firebase.firestore.test.ts
PASS  __tests__/firebase/auth/firebase.auth.test.ts
PASS  __tests__/app/speechContext.test.tsx
PASS  __tests__/screens/login/login.screen.test.tsx
PASS  __tests__/components/components.test.tsx
PASS  __tests__/components/topHeader.test.tsx
PASS  __tests__/screens/static/static.pages.test.tsx
PASS  __tests__/components/guestGate.test.tsx
PASS  __tests__/screens/signup/signup.screen.test.tsx
PASS  __tests__/screens/music/music.screen.test.tsx
PASS  __tests__/screens/hub/hub.screen.test.tsx
PASS  __tests__/screens/ai/ai.screen.test.tsx
```

Figure 5: Web Test Files

```
Test Suites: 18 passed, 18 total
Tests:       79 passed, 79 total
Snapshots:   0 total
Time:        3.996 s
Ran all test suites.
```

Figure 6: Web Total Tests

File		Statements	Branches	Functions	Lines				
src		91.66%	11/12	100%	9/9	100%	1/1	100%	11/11
src/app		72.72%	16/22	100%	0/0	66.66%	2/3	80%	16/20
src/app/ai		90.47%	76/84	81.48%	44/54	100%	11/11	93.58%	73/78
src/app/auth/login		86.48%	32/37	57.14%	8/14	100%	5/5	85.71%	30/35
src/app/auth/signup		89.36%	42/47	63.63%	14/22	100%	7/7	88.37%	38/43
src/app/components		90.47%	19/21	84.61%	11/13	100%	2/2	90%	18/20
src/app/context		100%	10/10	100%	2/2	100%	2/2	100%	10/10
src/app/devices		100%	6/6	100%	0/0	100%	2/2	100%	6/6
src/app/emergency		95.06%	77/81	75%	12/16	100%	16/16	97.4%	75/77
src/app/hooks		87.3%	55/63	65%	13/20	83.33%	10/12	92.98%	53/57
src/app/hub		61.17%	115/188	44.69%	101/226	68.75%	22/32	65.64%	107/163
src/app/music		58.01%	257/443	45.84%	127/277	57.97%	40/69	59.03%	245/415
src/app/voice		100%	5/5	100%	0/0	100%	1/1	100%	5/5
src/components		80%	48/60	75%	21/28	73.91%	17/23	81.03%	47/58
src/lib		0%	0/6	100%	0/0	100%	0/0	0%	0/5
src/utlis		87.23%	41/47	58.33%	7/12	100%	9/9	100%	33/33

Figure 7: Web Coverage